Data Cleaning: Missing Value Detector (First Principles)

This module implements a custom missing-value scanner — built without pandas or any data library — to identify, quantify, and flag problematic columns in a raw dataset.

Why build this from scratch?

Real-world data is rarely clean. Missing values don't always arrive as a tidy `None` or `NaN` — they show up as empty strings, placeholder text, or sentinel numbers that only make sense in context.

Building this manually forces three things pandas `.isnull()` doesn't:

- **Custom null definitions** — `""`, `"N/A"`, `"unknown"`, `-999`, and `0` can all mean *missing*, depending on the domain. This scanner lets you define what "null" means for your data, not the library's default.
- **Missingness density mapping** — not just *how many* nulls exist, but *where* they're concentrated. A column with 60% missing values needs a different strategy than one with 2%.
- **Imputation readiness** — the output feeds directly into the next pipeline step: deciding whether to drop, fill with mean/median, or flag for manual review.

The logic

For each column, the scanner calculates a **Missingness Ratio**:

$$\text{Missing \%} = \left(\frac{\text{Null Count in Column}}{\text{Total Rows}} \right) \times 100$$

If the ratio exceeds a defined threshold (default: 20%), the column is flagged for manual review rather than automatic imputation — because blindly filling 60% of a column with a mean is not cleaning data, it's manufacturing it.

Implementation

```python
```

```python
def find_missing(rows, null_values=None, threshold=20.0):
    """
    Scans a list of dicts and returns per-column missingness stats.

    Args:
        rows        : list of dicts (your dataset)
        null_values : custom definitions of 'missing' (default: None, "", "N/A",
        threshold   : missingness % above which a column is flagged (default: 20

    Returns:
        dict with count, percentage, and flag per column
    """
    if null_values is None:
        null_values = {None, "", "N/A", "n/a", "unknown", -999}

    if not rows:
        return {}

    columns = rows[0].keys()
    total_rows = len(rows)
    result = {}

    for col in columns:
        null_count = sum(1 for row in rows if row.get(col) in null_values)
        pct = (null_count / total_rows) * 100
        result[col] = {
            "missing_count": null_count,
            "missing_pct": round(pct, 2),
            "flagged": pct > threshold
        }

    return result
```

Example output

python

```python
data = [
    {"name": "Alice", "age": 30,    "city": "Delhi"},
    {"name": "Bob",   "age": None,  "city": ""},
    {"name": "Carol", "age": 28,    "city": None},
    {"name": "Raam",  "age": -999,  "city": "N/A"},
]

print(find_missing(data))
```

```
{
  "name": {"missing_count": 0, "missing_pct": 0.0,   "flagged": False},
  "age":  {"missing_count": 2, "missing_pct": 50.0,  "flagged": True},
  "city": {"missing_count": 3, "missing_pct": 75.0,  "flagged": True}
}
```

> `age` and `city` are both flagged — 50% and 75% missingness respectively. Neither should be auto-imputed without a deliberate decision.

What this maps to in the real DS stack

This implementation	pandas / sklearn equivalent
`row.get(col) in null_values`	`df.isnull()` + custom `na_values` in `pd.read_csv()`
Missingness ratio per column	`df.isnull().mean() * 100`
Threshold flagging	Manual review step before `SimpleImputer`

Understanding this logic is what makes `df.isnull().sum()` meaningful rather than mechanical.

Part of the vg-portfolio — every concept earned, not installed.